

2026年04月10日

迷宫路径搜索算法 对比分析

BFS、DFS、Dijkstra、A* 及扩展算法

汇报人：唐一嘉
10245101434

目录

01 | 问题定义与算法概览

02 | 拓展对比分析

03 | 搜索路径可视化分析

04 | 结论

01

问题定义与算法概览

问题定义：迷宫寻路



$n \times m$

迷宫维度规模



4 向

上下左右移动



1 单位

每步移动代价



问题定义

迷宫问题

无测评机，需要Presentation

求解

分别用BFS, DFS, Dijkstra, A star解决迷宫问题

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含0或1，其中0表示可以走的路，1表示不可通过的墙壁。

最初，有一个人位于左上角 $(1, 1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n, m) 处，至少需要移动多少次。

数据保证 $(1, 1)$ 处和 (n, m) 处的数字为0，且一定至少存在一条通路。

输入格式

第一行包含两个整数 n 和 m 。

接下来 n 行，每行包含 m 个整数（0或1），表示完整的二维数组迷宫。

输出格式

输出一个整数，表示从左上角移动至右下角的最少移动次数。

数据范围

$1 \leq n, m \leq 100$



问题本质特征

- 1 等权图模型: 每步移动代价恒为1，符合无权图特性
- 2 最短路径: 转化为图论中的单源最短路径问题
- 3 网格拓扑: 隐式图结构，邻接关系由空间位置确定

四大核心算法复杂度分析

DFS

Depth First Search

- **时间复杂度: $O(V+E)$**
遍历所有顶点与边, 线性时间效率。
- **空间复杂度: $O(h)$**
 h 为递归深度, 仅需栈空间, 内存占用最小。
- **实现特点**
栈结构实现, 不保证最优路径, 适合解的存在性判断。

栈实现 | 空间最优 $O(h)$

BFS

Breadth First Search

- **时间复杂度: $O(V+E)$ 。**
- **空间复杂度: $O(V)$**
需队列存储全量节点及访问标记数组。
- **最优性保证**
无权图中保证找到最短路径, 层序扩展特性。

队列实现 | $O(V+E)$

Dijkstra

Shortest Path

- **时间复杂度: $O((V^2)\log V)$**
优先队列 (数组) 插入删除开销 $\log V$ 。
- **空间复杂度: $O(V)$**
距离数组、优先队列及访问标记存储。
- **适用场景**
贪心策略, 适用于带权图单源最短路径求解。在无权图中退化为BFS。

贪心策略 | 优先队列 $O(\log V)$

A*

A Star Search

- **时间复杂度: $O((V+E)\log V)$**
启发式排序优先队列维护开销。
- **空间复杂度: $O(V)$**
Open/Closed集合存储已访问和待访问节点。
- **启发式优势**
 $f=g+h$ 评估函数引导, 搜索效率高。

启发搜索 | 效率较优 $f=g+h$

算法概览与复杂度对比

对比维度	DFS	BFS	Dijkstra	A*
 时间复杂度	$O(V+E)$	$O(V+E)$	$O(V^2\log V)$	$O((V+E)\log V)$
 空间复杂度	$O(h)$	$O(V)$	$O(V)$	$O(V)$
 最优解保证	非最优	最优	最优	最优

普通的DFS无法找到最短路径！

为此，需要在普通的DFS算法上进行修改，
实现一个修改版的DFS (Powerful DFS)：

1. 使用递归 DFS 枚举所有从起点到终点的路径
2. 使用回溯 (backtracking) 避免当前路径中的循环
3. 比较所有路径长度，返回最短的那条

优化：

剪枝1：当当前遍历探索到了某个节点，但它发现此刻的步长 (cost) 大于、或者等于我们历史上曾经到达过该节点时的最优步长 ($\text{currentCost} \geq \text{prevBestCost}$)，那么我们就直接将当前分支断掉。

剪枝2 (if (currentPath.length $\geq$ minLength) return;): 在找到某个可行解之后，不再搜索比它还长的不优路径直接剪枝，完全符合穷举的底层原理而不是通过猜测未来 (启发式) 来找解。

实验设计与环境



迷宫环境参数

核心

采用100行×100列的矩形迷宫网格，墙壁密度设置为随机生成。起点与终点分别位于迷宫中固定点，确保路径搜索具有足够的复杂度和挑战性，模拟真实场景中的障碍物分布。

 网格规模: 10000个节点



三项评估指标体系

关键

建立多维度评估框架：访问节点数衡量搜索效率（越少越好），路径长度评估解的质量（越短越好），运行时间反映实际性能（毫秒级）。三项指标综合评判算法优劣。

 性能基准: 实时毫秒级记录



实验方法与可视化工具

支撑

控制变量法：在同一迷宫实例上运行所有对比算法，确保环境一致性。基于前端技术栈开发可视化平台，支持算法执行过程的实时动画演示与数据面板展示，便于直观对比分析。

 演示模式: 实时路径渲染

02

拓展对比分析

经典DFS vs 完全DFS

经典DFS

Standard DFS



1. 起始节点开始搜索

从起点初始化栈，标记访问状态



2. 深度优先遍历路径

优先访问深层节点，回溯时继续探索



3. 找到终点立即停止

首次到达目标节点即终止算法



4. 返回当前路径

输出当前栈中记录的遍历路径

🕒 时间: $O(V+E)$

📦 空间: $O(h)$

🏔️ 路径: 非最优

⚡ 速度: 极快

PowerfulDFS



1. 遍历所有可能节点

不提前终止，探索全图所有可达节点



2. 松弛操作更新最短距离

比较当前路径与已知最短路径，保留最优



3. 返回最短路径

遍历完成后输出全局最优路径

🕒 时间: $O(V^2)$

📦 空间: $O(V)$

🏔️ 路径: 最优解

🐢 速度: 极慢

BFS vs Bidirectional BFS



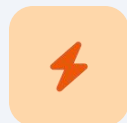
单向 BFS
搜索半径 r



双向 BFS

双半径 $r/2$

搜索空间 $\pi r^2/2$, 减少约50%



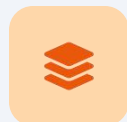
时间复杂度
 $O(V + E)$



时间复杂度

$O(V/2 + E/2)$

理论速度提升 2 倍



节点访问
单向波扩展



节点访问

双向波相遇

中间相遇, 访问节点数显著减小

A* vs Weighted A* vs Greedy



A* Search

最优路径保证

$$f(n) = g(n) + h(n)$$

- ✓ **评估策略**: 实际代价与启发估计平衡
- 最优性**:
- ✓ 保证找到最短路径 (可采纳启发式)
- ✓ **搜索范围**: 遍历节点较多, 探索全面
- ✓ **适用场景**: 路径质量要求严格的应用



带权重的A*Search

不保证最优路径

$$f(n) = g(n) + w \cdot h(n)$$

- ✓ **评估策略**: 实际代价与启发估计平衡, 通过权重系数偏向启发式
- ✓ **最优性**: 不保证最优, 可能局部最优
- ✓ **搜索范围**: 访问节点较少, 搜索较激进
- ✓ **适用场景**: 实时性要求高的快速寻路



Greedy Search

纯启发式优先

$$f(n) = h(n)$$

- ✓ **评估策略**: 仅依赖启发函数估计
- 最优性**: 不保证最优, 可能局部最优
- ✓ **搜索范围**: 访问节点最少, 搜索激进
- ✓ **适用场景**: 实时性要求高的快速寻路

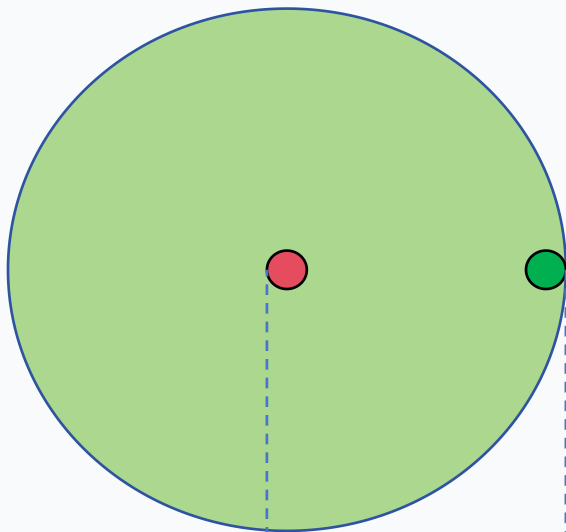
⚡ Weighted A* 折中方案

评估函数 $f(n) = g(n) + w \cdot h(n)$ ($w > 1$), 通过权重系数偏向启发式, 显著加速搜索同时保持路径质量在可接受范围内。

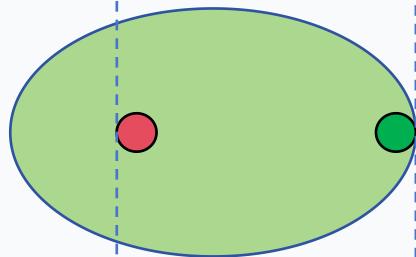
03

搜索路径可视化分析

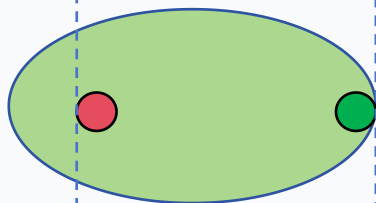
BFS:



A star:



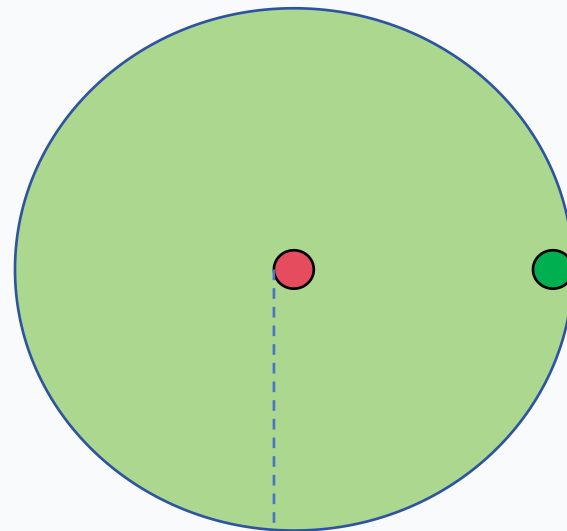
weighted A star:



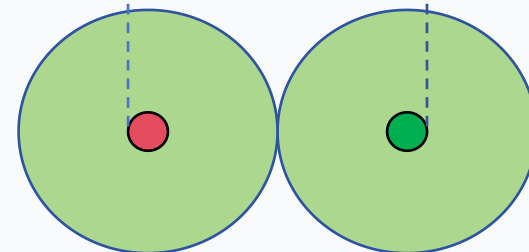
Greedy First Search:



BFS:



双向BFS:



04

结论

关键发现与结论



等权图最优解

- BFS是等权图最优选择，保证最短路径
- Bidirectional BFS理论上性能提升**2-4倍**，但实际测试中**不一定能达到**
- Dijkstra存在性能陷阱，资源浪费**20-100倍**



启发式算法平衡

- A*算法平衡最优性与执行效率
- Weighted A*更快但牺牲最优性
- Greedy Best-First Search 同理



DFS路径局限

- DFS不保证找到最短路径
- PowerfulDFS搜索效率极低



决策选择指南

- 需要无权图最短路径：选BFS
- 有权复杂场景：选A*、Dijkstra算法

THE END

谢谢观看

汇报人：唐一嘉